

Author : Christophe Garion <garion@isae-supaero.fr>
Public : N7 2SN
Date : 22/05/26



Il est interdit d'utiliser votre téléphone portable ou votre ordinateur personnel durant l'examen. Il est également interdit d'utiliser un LLM comme Copilot, ChatGPT, Claude, Gemini etc durant l'examen.

Vous ne devez utiliser que les constructions vues en cours ou celles introduites explicitement dans le sujet. En particulier, les constructions pouvant être utilisées en programmation par contraintes comme `fd_all_different` ne sont pas autorisées.

Les réponses aux questions autres que du code seront insérées sous forme de commentaires dans le code Prolog (utilisation de `/**/` comme en C). N'hésitez pas également à mettre sous forme de commentaires les résultats de vos tests.

Résumé

Le but de ce BE est d'utiliser la programmation logique et le langage Prolog pour résoudre un problème. On s'intéresse ici à établir quelques statistiques sur les nombres premiers.

1 Présentation du problème

Les nombres premiers ont parfois des propriétés surprenantes. Si l'on prend deux nombres premiers consécutifs et que l'on regarde leurs derniers chiffres d_1 et d_2 (en base 10), on pourrait s'attendre qu'il n'y ait pas de différence de fréquence d'apparition entre les différentes valeurs de d_2 étant fixé d_1 . Mais ce n'est pas le cas. Par exemple, si on considère un nombre premier qui finit par 9, il y a presque 4 fois plus de chances que le nombre premier suivant finisse par un 1 que par un 9. L'objectif de ce projet est d'afficher de telles statistiques sur un ensemble fini de nombres premiers.

Il s'agira de :

- développer un prédicat permettant de calculer la liste des nombres premiers parmi les n premiers entiers ;
- développer un prédicat permettant de calculer les statistiques de « transition » d'un chiffre à l'autre parmi les nombres premiers consécutifs.

2 Rappels sur la syntaxe Prolog

Comme nous allons manipuler des entiers, voici un rappel de quelques opérateurs sur les entiers qui vous seront utiles et qui ont une syntaxe différente de celle habituellement utilisée dans les langages informatiques.

<code>mod</code>	modulo
<code>:=</code>	égalité
<code>=\=</code>	non égalité
<code>=<</code>	inférieur ou égal

3 Fichier à compléter

Vous trouverez sur Moodle une archive contenant un répertoire avec le fichier à compléter. Dans celui-ci, des prototypes pour les prédicats à définir existent, il faut bien sûr les remplacer par vos définitions.



N'oubliez pas de mettre vos deux noms dans le fichier Prolog à compléter.

4 Questions

1. définir un prédicat `range/3` dont le but est de produire la liste des entiers successifs compris entre deux bornes (incluses).

Voici quelques exemples d'évaluation du prédicat que vous pourrez utiliser comme tests :

```
| ?- range(1, 6, L).
L = [1,2,3,4,5,6] ? ;
no
| ?- range(2, 2, L).
L = [2] ? ;
no
| ?- range(4, 3, L).
L = [] ? ;
no
```

2. définir un prédicat `remove_multiple/3` dont le but est d'enlever les multiples d'un nombre donné dans une liste. `remove_multiple(N, L, R)` sera vrai si `R` est identique à la liste `L` à laquelle on a enlevé tous les multiples de `N`.

Voici quelques exemples d'évaluation du prédicat que vous pourrez utiliser comme tests :

```
| ?- remove_multiple(2, [2, 7, 8, 4, 5], L).
L = [7,5] ? ;
no
| ?- remove_multiple(3, [3, 6], L).
L = [] ? ;
no
| ?- remove_multiple(1, [1, 2, 3, 4], L).
L = [] ? ;
no
| ?- remove_multiple(7, [3, 6, 4], L).
L = [3,6,4] ? ;
(1 ms) no
```

3. définir un prédicat `sieve/2` qui « crible » une liste d'entiers en enlevant les multiples du premier entier de la liste, puis les multiples du deuxième etc. Si `L` contient la liste des entiers de 2 à n , `sieve(L, R)` sera vrai si `R` contient la liste des nombres premiers inférieurs à n .

Voici quelques exemples d'évaluation du prédicat que vous pourrez utiliser comme tests :

```
| ?- sieve([], L).
L = []
yes
| ?- sieve([4, 2, 7, 8, 6], L).
L = [4,2,7] ? ;
no
| ?- sieve([2, 3, 4, 5, 6, 7, 8, 9, 10], L).
L = [2,3,5,7] ? ;
no
```

4. écrire un prédicat `count/4` qui va permettre de construire une liste de triplets (`N1`, `N2`, `N3`) où `N1` et `N2` sont deux nombres entiers et `N3` représente un nombre d'occurrences du couple (`N1`, `N2`).

`count(N1, N2, L, R)` va permettre d'incrémenter le nombre d'occurrence de $(N1, N2)$ « dans » L .
Voici quelques exemples d'évaluation du prédicat que vous pourrez utiliser comme tests :

```
| ?- count(1, 2, [], R).

R = [(1,2,1)] ? ;

no
| ?- count(1, 2, [(2, 4, 1)], R).

R = [(2,4,1),(1,2,1)] ?

no
| ?- count(1, 2, [(1, 2, 4), (2, 4, 1)], R).

R = [(1,2,5),(2,4,1)] ? ;

(1 ms) no
| ?- count(1, 2, [(2, 4, 1), (1, 2, 3)], R).

R = [(2,4,1),(1,2,4)] ?

no
```

5. écrire un prédicat `make_stat/2` tel que `make_stat(L, R)` sera vrai si
- L est une liste d'entiers
 - R est une liste contenant des triplets $(D1, D2, N)$ tels que N est le nombre de paires d'entiers consécutifs (n_1, n_2) de L telles que le dernier chiffre de n_1 est $D1$ et le dernier chiffre de n_2 est $D2$.

Voici quelques exemples d'évaluation du prédicat que vous pourrez utiliser comme tests :

```
| ?- make_stat([], L).

no
| ?- make_stat([1, 2, 3, 4], L).

L = [(3,4,1),(2,3,1),(1,2,1)] ?

yes
| ?- make_stat([2, 3, 5, 7, 11, 13, 17, 19], L).

L = [(7,9,1),(3,7,1),(1,3,1),(7,1,1),(5,7,1),(3,5,1),(2,3,1)] ?

yes
| ?- make_stat([22, 42, 62, 71, 46, 28, 36, 36, 78], L).

L = [(6,8,2),(6,6,1),(8,6,1),(1,6,1),(2,1,1),(2,2,2)] ?

yes
```

6. vous pouvez ensuite exécuter la requête `prime_conspiracy(10000)` pour avoir les statistiques sur les nombres premiers inférieurs à 10000.

```
| ?- prime_conspiracy(10000).
1 -> 1: 41
1 -> 3: 104
1 -> 7: 123
1 -> 9: 38
2 -> 3: 1
3 -> 1: 53
3 -> 3: 32
3 -> 5: 1
3 -> 7: 99
3 -> 9: 124
```

```
5 -> 7: 1
7 -> 1: 90
7 -> 3: 89
7 -> 7: 30
7 -> 9: 99
9 -> 1: 122
9 -> 3: 84
9 -> 7: 55
9 -> 9: 42
```

5 Documents à rendre

Vous devez déposer pour le **22 mai 2026 à 17h00** sur le dépôt Moodle du cours le source d'un programme Prolog répondant aux questions ci-dessus. **Le fichier source devra impérativement s'appeler `prime-conspiracy-login1-login2.pl` où `login1` et `login2` sont les logins de votre binôme.** Les réponses aux questions autres que du code devront y être insérées sous forme de commentaires.

License CC BY-NC-SA 3.0



This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported license (CC BY-NC-SA 3.0)

You are free to Share (copy, distribute and transmit) and to Remix (adapt) this work under the following conditions:



Attribution – You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).



Noncommercial – You may not use this work for commercial purposes.



Share Alike – If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

See <http://creativecommons.org/licenses/by-nc-sa/3.0/> for more details.